Modern CNN Pipelines and Transfer Learning

Blocks, Scaling, Residuals, and Reusing Backbones

Prof. Nipun Batra

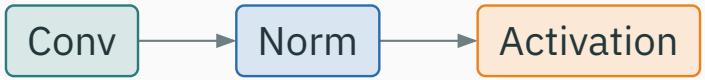
ES 667 · Deep Learning · IIT Gandhinagar

Why another CNN lecture?

Where we are

Last lecture built the **convolution** from scratch: locality, weight sharing, hierarchy. Our images are tensors, and the **primitive** we assemble networks from is one small block:

$$X \in \mathbb{R}^{H \times W \times C}, \qquad \text{Conv} \to \text{Norm} \to \text{Activation}$$



we know **how one block works** — today is about **how to design systems of them**

Today’s question

CNN **basics**: how does a convolution work?

Today: how do we design CNN **systems** that actually work?

The raw conv layer is the transistor. This lecture is the **circuit design**: stacking blocks into **backbones** through depth, multi-scale branches, residual connections, efficient factorizations — then **reusing** those backbones on new tasks.

The arc of this lecture

1. **VGG** — depth from stacking small kernels;
2. **1×1 conv** — mixing channels cheaply;
3. **Inception** — parallel multi-scale branches;
4. **ResNet** — residual connections that keep deep nets trainable;
5. **Efficient CNNs** — depthwise-separable, compound scaling, ConvNeXt;
6. **Pipeline + transfer** — reusing pretrained backbones on your data.

VGG — depth from simplicity

One design principle

VGG asked: what if we **only** ever use small 3×3 convs, and just go **deep**?

- every conv is 3×3, stride 1, pad 1 — so spatial size is **preserved**;
- downsampling is left entirely to 2×2 **max-pooling**;
- stack 16–19 such layers into a uniform, repetitive network.

one kernel size, one stride — depth is the only knob

Why so many 3×3 kernels?

D DERIVATION

Stacking small kernels **grows the receptive field** while adding **nonlinearities**:

- two 3×3 convs see a 5×5 input region;
- three 3×3 convs see a 7×7 region;

$$r_\ell = r_{\ell-1} + (k-1), \qquad 1 \overset{3\times3}{\to} 3 \overset{3\times3}{\to} 5 \overset{3\times3}{\to} 7$$

same field as one big kernel — but with **more** ReLUs in between, so a richer function

Two small beat one big

D DERIVATION

Cover a 5×5 field two ways, and count parameters (channels C in and out):

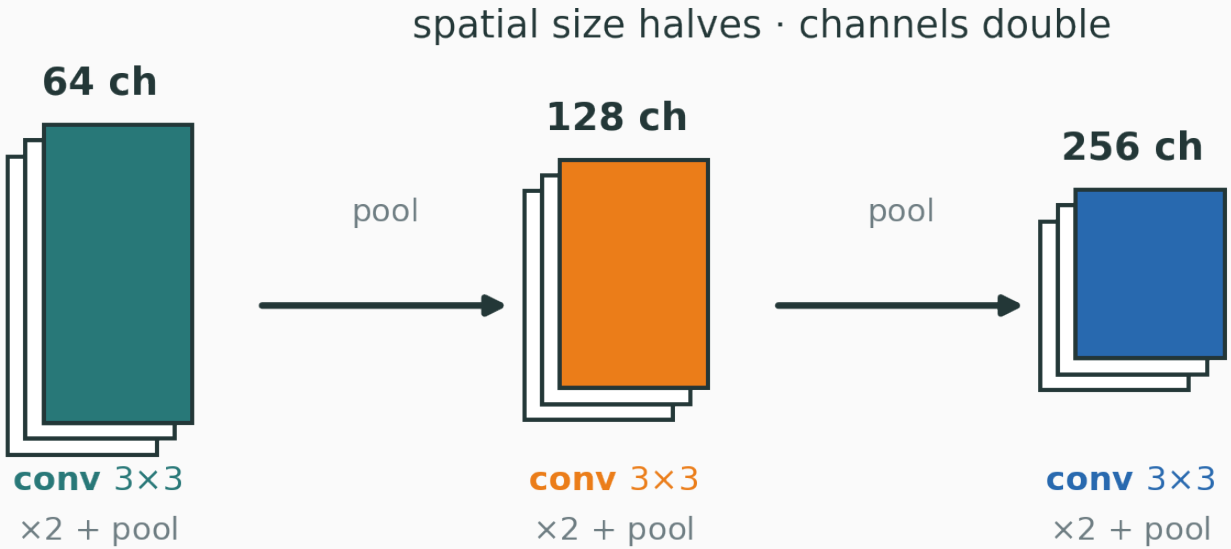
	two 3×3	**one 5×5**
receptive field	5×5	5×5
parameters	$2 \cdot 3 \cdot 3C^2 = 18C^2$	$5 \cdot 5C^2 = 25C^2$
nonlinearities	2	1

$18C^2 < 25C^2$ **— fewer parameters and more nonlinearity**

The VGG stage

V VISUAL

A stage = a couple of 3×3 convs, a pool, and **double** the channels:



$64 \to 128 \to 256 \to 512$ channels as spatial size halves — resolution traded for semantics

What VGG taught us

- **depth matters** — a deep stack of simple blocks beats a shallow clever one;
- **small kernels are enough** — no need for large, exotic filters;
- **uniform blocks** make networks easy to design, scale, and reason about.

But VGG is **parameter-heavy**: its fully-connected head alone holds over 100 million weights. The next ideas are largely about **removing that waste**.

Interactive: stacking small kernels

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/receptive-field-grower

Add 3×3 conv layers one at a time and watch the receptive field grow $3 \to 5 \to 7 \to \ldots$ — the exact mechanism behind VGG's "go deep with small kernels".

Q. How many stacked 3×3 convs match the receptive field of one 9×9 kernel?

The 1×1 convolution

A convolution with no neighbours

Take a conv whose kernel is a single pixel. At each location (h, w):

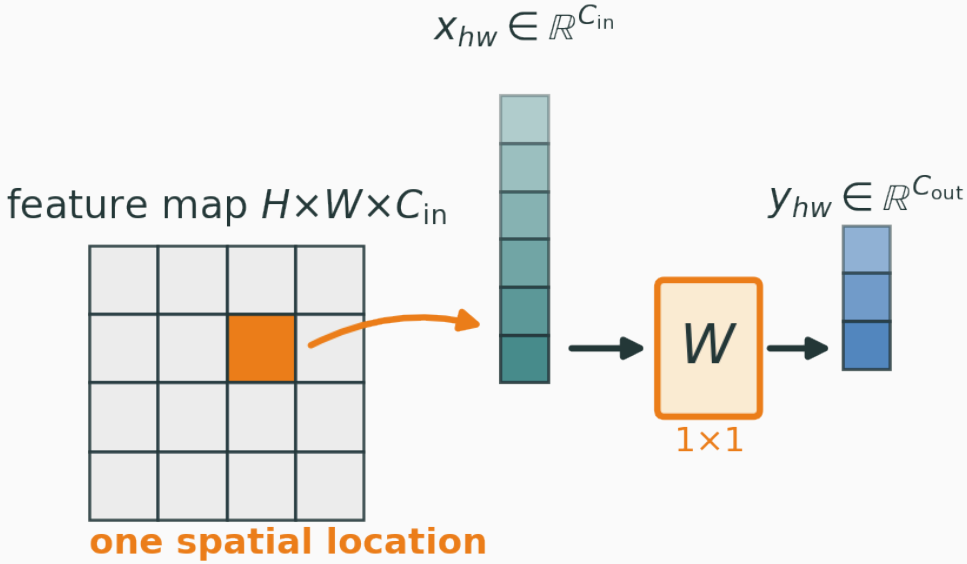
$$y_{hw} = W\, x_{hw} + b, \qquad x_{hw} \in \mathbb{R}^{C_{\text{in}}} \to y_{hw} \in \mathbb{R}^{C_{\text{out}}}$$

- it looks at **one** pixel, across **all** its channels;
- it **mixes channels**, but touches **no spatial neighbours**;
- W is just a $C_{\text{out}} \times C_{\text{in}}$ matrix, shared over every position.

a 3×3 asks "what pattern is **around** here?"; a 1×1 asks "what **combination** of channels is here?"

1×1 is a per-pixel MLP

V VISUAL



$y_{hw} = W x_{hw} + b$ — mixes channels, not neighbours

the **same** tiny MLP W is applied independently at every spatial location

What a 1×1 conv is for

Use	What it does
change channel dim	project $C_{\text{in}} \to C_{\text{out}}$ (grow or shrink)
bottleneck	squeeze channels before an expensive conv
feature mixing	recombine channels into new features
residual projection	match shapes on a skip connection
segmentation head	per-pixel class scores

Worked example: 1×1 vs 3×3

D DERIVATION

Project a feature map $H \times W \times 256 \to H \times W \times 64$. Count the weights:

$1{\times}1{:}\quad 1 \cdot 1 \cdot 256 \cdot 64 + 64 = 16{,}448$

$3{\times}3{:}\quad 3 \cdot 3 \cdot 256 \cdot 64 + 64 = 147{,}520$

$147{,}520 \;/\; 16{,}448 \approx 9\times$

same channel change, $9\times$ fewer parameters

if you only need to **re-mix channels**, spending a 3×3 on it is wasteful

The bottleneck idea

Instead of a fat 3×3 that maps $256 \to 256$, **sandwich** it between two 1×1s:

$$\underbrace{1 \times 1}_{256 \to 64} \quad \to \quad \underbrace{3 \times 3}_{64 \to 64} \quad \to \quad \underbrace{1 \times 1}_{64 \to 256}$$

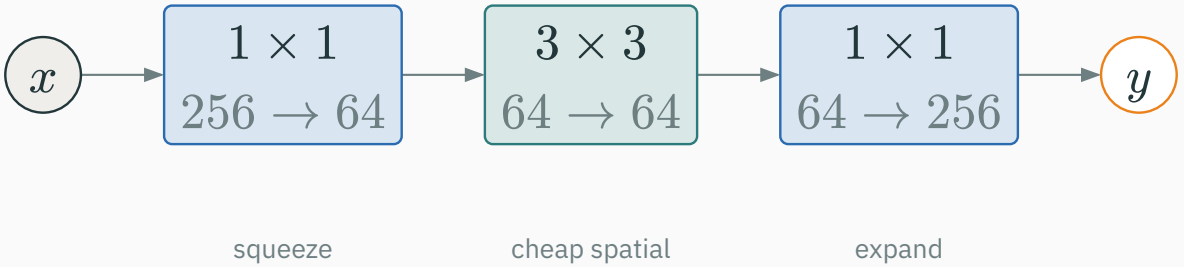
- the 1×1s **squeeze** then **restore** the channel count;
- the expensive 3×3 runs in the **cheap** 64-channel space;
- far less compute, with similar representational power.

This **bottleneck** is the engine inside ResNet, Inception, and MobileNet — reduce channels, do the spatial work, expand back.

The bottleneck, visually

V VISUAL

Two 1×1s wrap the costly 3×3, so the spatial work runs in a **thin** channel space:



squeeze $256 \to 64$, do the 3×3 in the cheap 64-channel space, then expand $64 \to 256$

Inception — multi-scale, factorized

Which kernel size? All of them

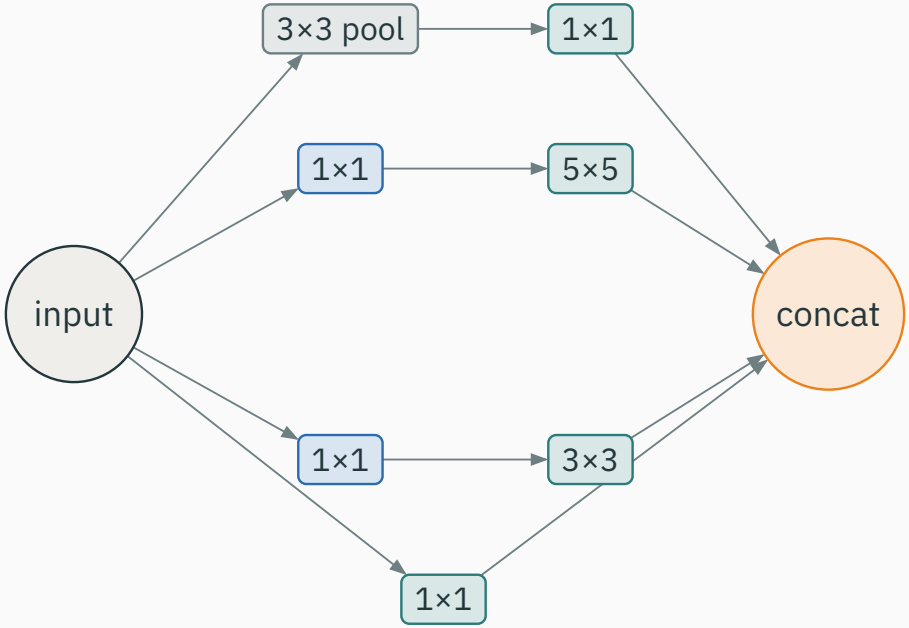
Objects appear at **many scales** — an eye is tiny, a face is large. Why pick one kernel?

- run several kernel sizes **in parallel**: 1×1, 3×3, 5×5, and a pool;
- **concatenate** their outputs along the channel axis;
- let training decide how much to lean on each scale.

don’t choose a receptive field — offer several and combine

The Inception module

V VISUAL



four parallel branches → concat channels · note the 1×1 **reductions** before the costly convs

Why the 1×1 reductions?

D DERIVATION

A 5×5 conv straight onto $C = 256$, producing 128 channels, is expensive:

$$\text{direct } 5{\times}5: \quad 5 \cdot 5 \cdot 256 \cdot 128 = 819{,}200$$

Reduce channels **first** with a 1×1 ($256 \to 32$), then do the 5×5 ($32 \to 128$):

- 1×1 reduce: $256 \cdot 32 = 8{,}192$;
- 5×5 on the thin map: $25 \cdot 32 \cdot 128 = 102{,}400$;

$$8{,}192 + 102{,}400 = 110{,}592$$

$\approx 7.4 \times$ **cheaper — same output shape**

The Inception lesson

- **multi-scale in parallel** — capture fine and coarse structure in one block;
- 1×1 **reductions** — put the expensive spatial convs in a low-channel space;
- **put computation where it matters** — spend FLOPs on spatial work, not channel bulk.

Inception is Part III's bottleneck idea, applied **per branch**. The recurring move: shrink channels, do spatial work, then combine.

Interactive: cost of a block

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/inception-cost

drag the 1×1 reduction width and watch a branch's FLOPs collapse, then rise again as you widen the reduced space.

Q. If the 1×1 reduced $256 \to 64$ instead of $256 \to 32$, would the branch be cheaper or dearer than the direct 5×5?

ResNet — the residual backbone

Deeper should not be worse

Add layers to a working network. In principle the extra layers could learn the **identity** and do no harm.

Yet very deep **plain** nets trained **worse** — higher training error than shallower ones. This is not overfitting; the optimizer simply **cannot fit** them. Deep plain stacks are hard to optimize.

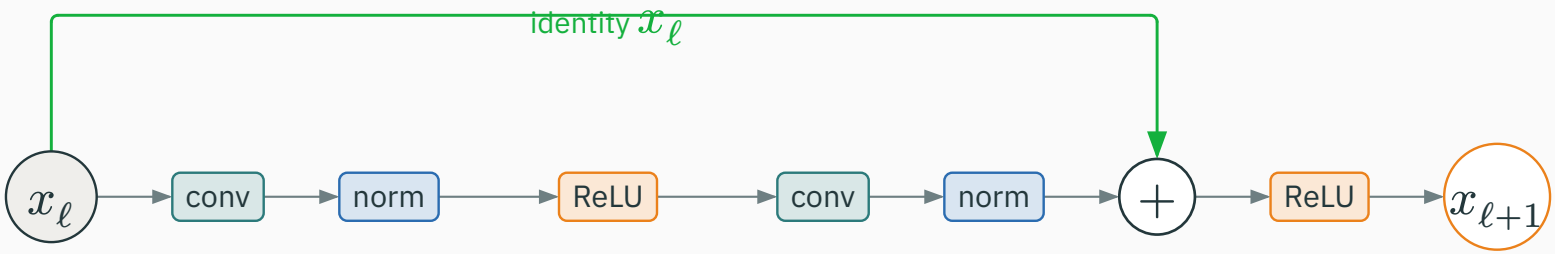
ResNet's fix: make it easy for a block to represent "leave the input alone"

The residual block

V VISUAL

Instead of learning a full mapping $H(x)$, learn a **residual** $F(x)$ added to the input:

$$x_{\ell+1} = x_\ell + F_\ell(x_\ell)$$



the green **identity** path carries x_ℓ straight to the sum — the block only learns the **correction**

Why residuals train

D DERIVATION

Look at the gradient flowing back through one block. Since $x_{\ell+1} = x_\ell + F_\ell(x_\ell)$:

$$\frac{\partial \mathcal{L}}{\partial x_\ell} = \frac{\partial \mathcal{L}}{\partial x_{\ell+1}} \cdot \left(I + \frac{\partial F_\ell}{\partial x_\ell} \right)$$

- the I term is a **direct highway**: gradient reaches x_ℓ **undiminished**;
- even if $\partial F_\ell / \partial x_\ell$ is tiny, the identity keeps the signal alive.

skip connections defeat vanishing gradients — the same trick that tamed deep MLPs

Basic vs bottleneck block

Two flavours of residual block trade depth-per-block against cost:

	basic	**bottleneck**
structure	$3 \times 3 \to 3 \times 3$	$1 \times 1 \to 3 \times 3 \to 1 \times 1$
1×1 role	—	reduce then expand channels
used in	ResNet-18/34	ResNet-50/101/152

The bottleneck block **is** the Part III sandwich: 1×1 squeeze → 3×3 spatial → 1×1 expand, wrapped in a skip connection.

When shapes change: projection shortcut

At a downsampling stage the spatial size halves and channels double — so x_ℓ and $F(x_\ell)$ **no longer match**. Fix the shortcut with a 1×1 conv (stride 2) that reshapes the identity:

$$x_{\ell+1} = P(x_\ell) + F(x_\ell), \qquad P = 1 \times 1 \text{ conv, stride } 2$$

the shortcut is **usually** the identity; only at shape changes does it become a learned 1×1 projection

ResNet as a backbone

V VISUAL

A ResNet is a stack of residual stages producing feature maps at **decreasing resolution**:

Stage	Resolution	Feeds
C2	$1/4$	fine detail — small objects
C3	$1/8$	mid-level parts
C4	$1/16$	larger context
C5	$1/32$	global semantics

one backbone → classification, detection, segmentation, FPN heads all tap these maps

Interactive: residual flow

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/resnet

Toggle the skip connection on and off and watch gradients flow (or vanish) through a deep stack — see the identity path keep the signal alive all the way back.

Efficient CNNs

The cost of a standard conv

A $k \times k$ conv from C_{in} to C_{out} channels over an $H \times W$ map costs:

$$\text{cost} = k^2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot HW$$

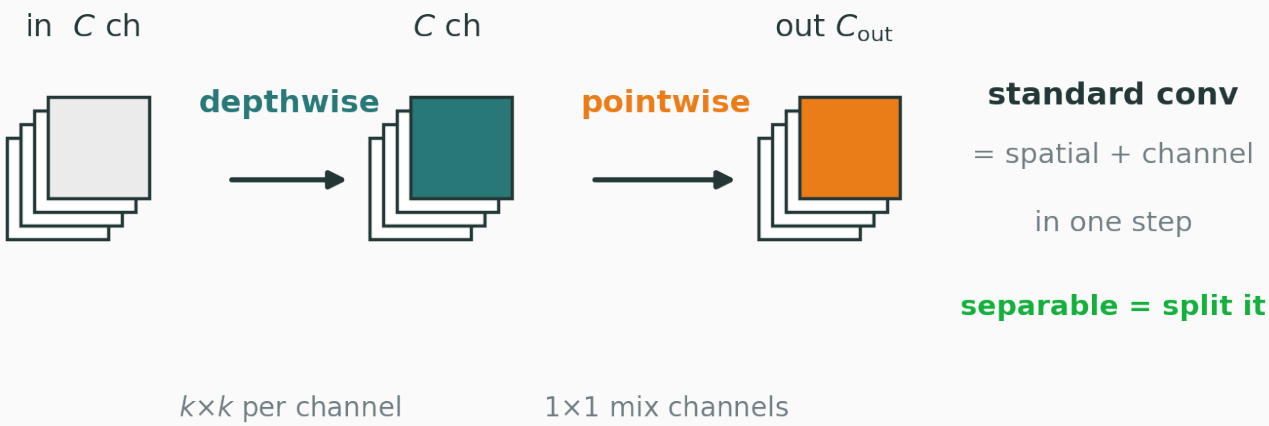
- one factor for the **spatial** window (k^2);
- one factor for **every input–output channel pair** ($C_{\text{in}} \cdot C_{\text{out}}$);
- that channel-pair product is where most of the cost hides.

idea: separate the spatial part from the channel-mixing part

Depthwise separable convolution

V VISUAL

Split one conv into two cheaper steps:



- **depthwise** — one $k \times k$ filter **per channel**: spatial only, cost $k^2 \cdot C_{\text{in}} \cdot HW$;
- **pointwise** — a 1×1 mixing channels: cost $C_{\text{in}} \cdot C_{\text{out}} \cdot HW$.

MobileNet's core block — the two factors **add** instead of **multiplying**

Worked cost: separable vs standard

D DERIVATION

A 3×3 conv with $C_{\text{in}} = C_{\text{out}} = 128$ (drop the shared HW factor):

$$\text{standard:} \quad 3^2 \cdot 128 \cdot 128 = 147{,}456$$

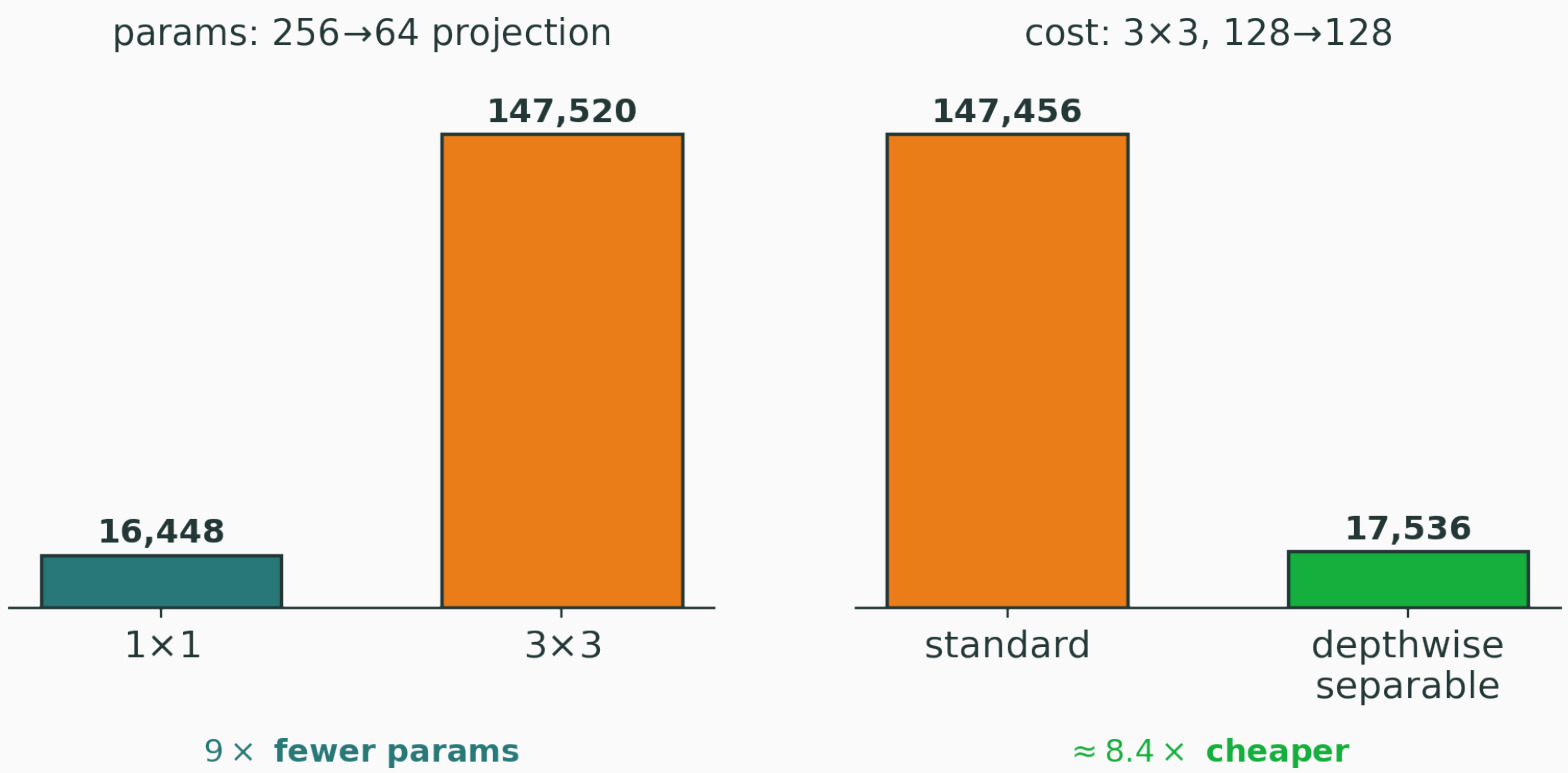
- depthwise — one 3×3 per channel: $3^2 \cdot 128 = 1{,}152$;
- pointwise — a 1×1 mixing channels: $128 \cdot 128 = 16{,}384$;

$$\text{separable total:} \quad 1{,}152 + 16{,}384 = 17{,}536$$

$$147{,}456 \;/\; 17{,}536 \approx 8.4 \times \textbf{cheaper}$$

Cost, side by side

V VISUAL



9 × **fewer params**

≈ 8.4 × **cheaper**

left: 1×1 vs 3×3 params · right: standard vs depthwise-separable cost

EfficientNet: compound scaling

You can make a net bigger three ways

EfficientNet: compound scaling

depth (more layers), **width** (more channels), **resolution** (bigger input).

- scaling **one** alone hits diminishing returns quickly;
- EfficientNet scales all three **together**, in a fixed ratio;

balanced growth gives more accuracy per FLOP than any single axis

ConvNeXt: a modernized ConvNet

★ OPTIONAL

Take a plain ResNet and apply every modern trick, one at a time:

- **larger** depthwise kernels (7×7); **inverted** bottlenecks;
- **fewer** activations and norms per block; **LayerNorm** in place of BatchNorm;
- modern training recipe (augmentation, schedules, AdamW).

No attention, no self-attention — just a carefully modernized **convolutional** net that competes with Transformer-era backbones. Convolution is far from obsolete.

The efficiency axes

Each idea in this lecture buys a **different** kind of efficiency:

Technique	**What it saves**
1×1 bottleneck	parameters + FLOPs
depthwise separable	FLOPs
global average pooling	parameters (no giant FC head)
residual connections	trainability (deep nets converge)
compound scaling	accuracy per FLOP
transfer learning	data + training time

The classification pipeline

The modern pipeline

V VISUAL

Training an image classifier today is an **assembly line** of standard parts:

data → augment → **pretrained backbone** → head → loss → optimizer → scheduler → validate → checkpoint

Preprocessing: match the backbone

Standardise each channel with a fixed mean and standard deviation:

$$x' = \frac{x - \mu}{\sigma}$$

Use the **same** normalization the backbone was pretrained with (its ImageNet μ, σ). Feeding differently-scaled inputs to a pretrained net silently **breaks** its features.

a “model expects these exact stats” contract — the single most common transfer bug

The classification head

The backbone outputs a feature map; a tiny head turns it into class probabilities:

$$h_c = \frac{1}{H'W'} \sum_{i,j} F_{ijc} \qquad \text{(global average pool)}$$

$$z = Wh + b, \qquad p = \mathrm{softmax}(z), \qquad \mathcal{L} = -\log p_y$$

GAP (from Network-in-Network) collapses $H' \times W' \times C$ to a length-C vector — no giant flatten

Worked example: shapes through the head

D DERIVATION

Track shapes and count the head's parameters ($K = 10$ classes):

Stage	Shape	Note
input	$224 \times 224 \times 3$	RGB image
backbone F	$7 \times 7 \times 2048$	ResNet-50 output
global avg pool	2048	collapse H', W'
linear $\to K$	10	class scores

$$\text{head params} = 2048 \cdot 10 + 10 = 20{,}480 + 10 = 20{,}490$$

a 20k-parameter head on a 25-million-parameter backbone — the head is **tiny**

Transfer learning

Why it works

Early and middle layers learn **generic** visual features — edges, textures, parts — that recur across **every** vision task.

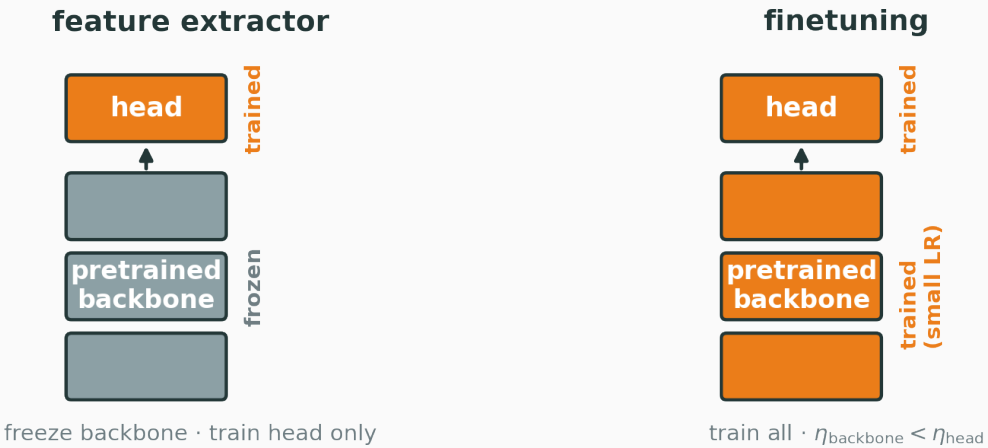
$$\theta_{\text{pretrained}} \xrightarrow{\text{adapt}} \theta_{\text{your task}}$$

- so **start** from weights pretrained on a large dataset (e.g. ImageNet);
- **keep** the reusable feature extractor; only the task-specific top must change.

reuse features, don't relearn them — strong results from little data

Mode 1: feature extractor

V VISUAL



Freeze the backbone, train **only** a new head.

- backbone weights are fixed — no gradient update;
- only the small head learns.

Good for **small data**, **similar classes**, or **limited compute**.

Mode 2: finetuning

Initialise from pretrained weights, then train **all** layers — but gently:

$$\eta_{\text{backbone}} < \eta_{\text{head}}$$

- the head starts random → needs a **larger** learning rate;
- the backbone is already good → a **small** rate nudges it without erasing it.

good for **larger** datasets or a **different domain** (medical, satellite, art) where features must shift

A safe recipe: probe, then finetune

1. **freeze** the backbone, train the head to convergence (a **linear probe**);
2. **unfreeze** the last block or two, continue with a small learning rate;
3. **optionally** unfreeze everything, using **discriminative** rates — lower for earlier layers.

Training the head first stops large early gradients from a random head **wrecking** the pretrained features on step one.

Worked example: how few new weights?

D DERIVATION

$5{,}000$ images, $K = 6$ classes, a ResNet-50 backbone. Replace its $2048 \to 1000$ head:

$$\text{new head:} \quad 2048 \cdot 6 + 6 = 12{,}288 + 6 = 12{,}294 \quad \text{parameters}$$

≈ 12k weights to fit — versus millions if you trained the whole net from scratch

with only 5k images, fitting 12k new parameters is usually the lower-variance starting point; end-to-end adaptation of a 25M pretrained backbone can still work, but needs a smaller LR and careful validation.

Transfer pitfalls

Pitfall	Symptom
wrong normalization	pretrained features silently degrade
finetune LR too high	catastrophic forgetting of good weights
forgot `train()`/`eval()`	BatchNorm / dropout misbehave
inconsistent augmentation	train/val mismatch
class imbalance	head collapses to majority class
frozen BatchNorm stats	wrong running stats on new domain

Interactive: transfer playground

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/transfer-learning-playground

slide the dataset size and choose freeze vs finetune, then watch accuracy trade off against overfitting.

Q. You have 200 labelled images in a domain close to ImageNet. Freeze or finetune?

From backbones to dense prediction

Why this belongs before detection

Detection and segmentation do not start over — they **reuse** the very backbones we just built.

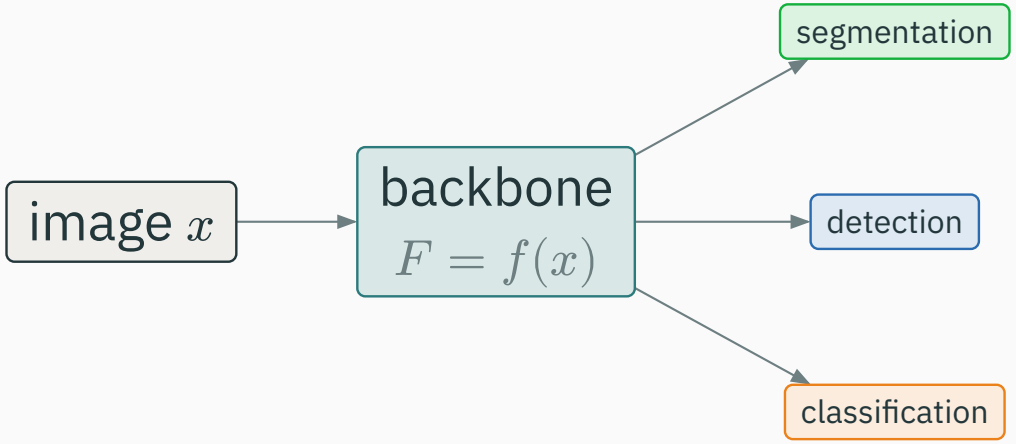
Classification
image → vector → **one** class label. The spatial map is **collapsed** away.

Dense prediction
image → **spatial** feature maps → boxes / masks. The spatial map is **kept**.

same feature extractor; the difference is entirely in the **head** and what it does with F

One backbone, many heads

V VISUAL



$$F = f_{\text{backbone}(x)}; \quad z = f_{\text{cls}(F)}; \quad \{b_i, c_i, s_i\} = f_{\text{det}(F)}; \quad M = f_{\text{seg}(F)}$$

the backbone is shared infrastructure; swap the **head** to change the task

Preview: feature pyramids

Objects come at **wildly** different scales — a distant sign, a nearby truck. One resolution cannot serve both.

- a Feature Pyramid Network (FPN) taps the backbone at **several** stages;
- it builds multi-resolution maps P_2, P_3, P_4, P_5 — fine to coarse;
- small objects use the fine maps; large objects use the coarse ones.

a brief preview — the detection lecture builds this properly on top of the C2–C5 maps we saw

Summary

The architectures at a glance

Architecture	Key idea
VGG	stack many small 3×3 convs — go deep
Inception	parallel multi-scale branches + 1×1 reductions
ResNet	residual learning — keep deep nets trainable
MobileNet	depthwise separable convs — cheap FLOPs
EfficientNet	compound scaling of depth/width/resolution
ConvNeXt	a modernized ConvNet, no attention needed

The design principles

1. **exploit local spatial structure** — convolutions, not full connections;
2. **share weights** across positions — one detector, used everywhere;
3. **preserve trainability** — residual connections and normalization;
4. **control compute** — 1×1 bottlenecks and separable convs;
5. **reuse pretrained backbones** — transfer beats training from scratch.

Final mental model — one idea

A raw conv is just a **feature detector**.
Modern CNNs turn it into a reusable **backbone**
through **blocks**, **scaling**, **residuals**, and **transfer**.

design the system, then reuse it — don’t train from scratch

Classification answers **what** is in the image.

Next: **object detection** — what is in the image, **and where**? — built on exactly these backbones.